Ex.No :01 **Show use of class in a c++**

Date :16.02.22

Aim:

To write a program to show use of class in a c++.

Algorithm:

Step 1: Start the program.
Step 2: Create class name as person.
Step 3: Declare variables name and age.
Step 4: Define the getdata function to get the values.
Step 5: Define the display() function to print the values.
Step 6: Create object for class name person.
Step 7: Call the function getdata(),display().
Step 8: Display the value.
Step 9: Stop the program.

Program:

```
#include<iostream>

using namespace std;
class person
{
char name[30];
int age;
public:
void getdata(void);
void display(void);
};
void person::getdata(void)
{
        cout<<"Enter name:";
        cin>>name;
        cout<<"Enter age:";
        cin>>age;
}
void person::display(void)
{
        cout<<"\n Name:"<<name;
        cout<<"\n age:"<<age;
}


int main()

        {
                person p;
                p.getdata();
                p.display();
                return 0;
        }
```

Output:

```
Enter name: Raja
Enter age : 18

Name: Raja
age: 18
```

Result:

Thus the program was compiled and executed successfully.

Ex.No :02 **Illustrate the use of inline function**.

Date :.18.02.22

Aim:

To write a c++ program to illustrate use of inline function.

Algorithm:

Step 1: Start the program.

Step 2:Create a class name as operation.

Step 3: Declare the variables integer, a, b, add, sub, mul, float div.

Step 4: Define the get()function to get the a and b value.

Step 5: Create a object for class name operation.

Step 6: Call the function sum(),get(),different(),product(),division().

Step 7: Display the value.

Step 8: Stop the program

Program:

```
#include<iostream>
Using namespace std;
Class operation
{
        int a,b,add,sub,mul;
        float div;
        public:
        void get();
        void sum();
void difference();
void product();
void division();
};
inline void operation::get()
{
        cout<<"Enter first value:";
        cin>>a;
        cout<<"Enter second value b:";
        cin>>b;
}
inline void operation::sum()
{
add=a+b;
cout<<"Addition of two numbers:"<<add<<"\n";
}
inline void operation::difference()
{
sub=a-b;
cout<<"Difference of two numbers:"<<sub<<"\n";
}
inline void operation::product()
{
mul=a*b;
cout<<"product of two numbers:"<<mul<<"\n";
}
inline void operation::division()
{
div=a/b;

cout<<"Division of two numbers:"<<div<<"\n";
}
```

```
int main()
{
        cout<<"Program using inline function \n";
operation s;
s.get();
s.sum();
s.difference();
s.product();
s.division();
return 0;
}
```

Output:

```
Program using inline function
Enter first value: 10
Enter second value b: 5
Addition of two numbers: 15
Difference of two numbers: 5
Product of two numbers:50
Division of two numbers: 2
```

Result:

Thus the program was compiled and executed successfully.

Ex.No: 03 **Illustrate function overloading**.

Date:

Aim:

To write a c++ program to illustrate function overloading.

Algorithm:

Step 1: Start the program

Step 2: Create a class name as funover.

Step 3: Define overloading function area(double r), area(int a),area (float l, float b).

Step 4: get value for calculation area of circle, area of square, area of rectangle.

Step 5: Print the value.

Step 6: Stop the program.

Program:

```
#include<iostream>
using namespace std;
classfunover
{
public:
double area(double r)
{
return(3.14*r*r);
}
int area(int a)
{
return (a*a);
}
float area(float l,float b)
{
return(l*b);
}
};
int main()
{
funover d;
double r;
int a;
floatl,b;
cout<<"\n Enter radius value for circle(double):";
cin>>r;
cout<<"\n Enter side value for square (int):";
cin>>a;
cout<<"\n Enter length value for rectangle(float):";
cin>>l;
cout<<"\n Enter breadth value for rectangle(float):";
cin>>b;
cout<<"\n Area of the circle is:"<<d.area(r)<<"sq.units";
cout<<"\n Area of the square is:"<<d.area(a)<<"sq.units";
cout<<"\n Area of the rectangle is:"<<d.area(l,b)<<"sq.units";
return 0;
  }
```

Output:

```
Enter radius value for circle(double):3
Enter side value for square (int)  :2
Enter length value for rectangle(float):5
Enter breadth value for rectangle(float):7

Area of the circle is:28.26sq.units
Area of the square is:4sq.units
 Area of the rectangle is:35sq.units
```

Result:

Thus the program was compiled and executed successfully.

Ex.No:04 **Illustrate the use of static data member**

Date:02.03.22

Aim:

To write a c++ program to illustrate the use of a static data member.

Algorithm:

Step 1: Start the program**.**

. Step 2:create a class name as student.

Step 3: Declare the variable introllno, char name,int marks.

Step 4: Define the student() function.

Step 5: Define the getdata() function to get the value.

Step 6: Define the putdata() function to print the value.

Step 7: Declare the static variable object for a class name as student.

Step 8: Display the value for total object created.

Step 9: Stop the program.

Program:

```
#include<iostream>
using namespace std;
class student
{
        private:
        int rollno;
        char name[10];
        int marks;
        public:
        static int objectcount;
  student()
    {
        objectcount++;
     }
void getdata()
  {
        cout<<"Enter roll number:"<<endl;
        cin>>rollno;
        cout<<"Enter name:"<<endl;
        cin>>name;
        cout<<"Enter marks:"<<endl;
        cin>>marks;
   }
void putdata()
{
cout<<"Roll number="<<rollno<<endl;
cout<<"name="<<name<<endl;
cout<<"marks="<<marks<<endl;
}
};
int student ::objectcount=0;
int main(void)
{
student s1;
s1.getdata()
s1.putdata();
student s2;
s2.getdata();
s2.putdata();
student s3;
s3.getdata();
```

```
s3.putdata();
cout<<"Total objeccreated="<<student::objectcount<<endl;
return 0;
}
```

Output:

```
Enter roll number: 212
Enter name : Vijay
Enter marks: 540
Roll number= 212
name= Vijay
marks= 540

Enter roll number: 213
Enter name: Vishal
Enter marks: 560
Roll number= 213
name= Vishal
marks= 560

Enter roll number: 214
Enter name: Surya
Enter marks: 470
Roll number= 214
name= Surya
marks= 470

Total objects created=3
```

Result:

Thus the program was compiled and executed successfully.

Ex.No: 05 **Illustrate the use of object arrays**

Date:09.03.22

Aim:

To write a c++ program to illustrate the use of object arrays.

Algorithm:

Step 1: Start the program.
Step 2: Create a class name as books.
Step 3: Declare the variable char title,float price.
Step 4: Define the getdata() function to get the variables.
Step 5: Define the putdata() function to print the variables.
Step 6: Create an array of objects.
Step 7: get the values using array of objects.
Step 8: put the values using array of objects.
Step 9: Stop the program.

Program:

```
#include<iostream>
using namespace std;
class books
{
char title[30];
float price;
public:
void getdata()
{
        cout<<"\n Title:";
        cin>>title;
        cout<<"\n price:";
        cin>>price;
}
void putdata()
{
        cout<<"\n Title:"<<title;
        cout<<"\n price:"<<price;
}
};
int main()
{
const int size=3;
int i;
books b[size];
for(i=0;i<size;i++)
{
        cout<<"\n Enter details book"<<(i+1);
        b[i].getdata();
}
for(i=0;i<size;i++)
{
        cout<<"\n Book"<<(i+1);
        b[i].putdata();
}
        return 0;
}
```

Output:

```
Enter details book1
 Title: c++
price: 457

Enter details book2
 Title: science
price: 798

Enter details book3
 Title: java
price:965

Book1
Title: c++
Price:457

Book2
Title:science
price:798

Book3
Title:java
price:965
```

Result:

Thus the program was compiled and executed successfully.

Ex.No: 06 **Use of friend function**

Date:11.03.22

Aim:

To write a c++ program to use friend function.

Algorithm:

Step 1: Start the program.

Step 2: Create a class name as student.
Step 3: Declare the variables introllno, char name, char dept.
Step 4: Define the get()function to get the variables.
Step 5: Define the put() using as a friend function.
Step 6: Create a object for class name student.
Step 7: Call the functions to print the values.
Step 8: Stop the program.

Program:

```
#include<iostream>
using namespace std;
class student
{
private:
        int rno;
        char name[10];
        char dept[10];
        public:
 void get()
  {
        cout<<"\n Enter the roll number:";
        cin>>rno;
        cout<<"\n Enter the name:";
        cin>>name;
        cout<<"\n Enter the department:";
        cin>>dept;
  }
        friend void put(student s);
};
void put(student s)
{
        cout<<"\n Roll number:"<<s.rno;
        cout<<"\n Name:"<<s.name;
        cout<<"\n Department:"<<s.dept;
}
int main()
{
        student obj;
        obj.get();
        put(obj);
        return 0;
}
```

Output:

```
Enter roll number: 212
Enter name: Ramesh
Enter marks: 520

Roll number= 212
name= Ramesh
marks= 520
```

Result:

Thus the program was compiled and executed successfully.

Ex.No: 07 Demonstrate the passing of arguments to the constructed function.

Date:16.03.22

Aim:

To write a c++ program to demonstrate the passing of argument to the constructor function.

Algorithm:

Step 1: Start the program.

Step 2: Create a class name as cons.

Step 3: Declare the variables integer x and y.

Step 4: Define the constructor function as const(int a,int b).

Step 5: Define display() function.

Step 6: Call the constructor function using arguments and print the values.

Program:

```
#include<iostream>
using namespace std;
class cons
{
int x,y;
public:
cons (int a,int b)
{
        x=a;
        y=b;
}
void display()
{
        cout<<"The value of x="<<x<<"and y="<<y;
}
    };
int main()
  {
   cons c1(12,13);
    c1.display();
    return 0;
  }
```

Output:

The value of x=12and y=13

Result:

Thus the program was compiled and executed successfully.

Ex.No: 08 **Show the how unary minus operator is overloaded**

Date:18.03.22

Aim:

To write a c++ program to show the unary minus operater overloaded.

Algorithm:

Step 1: Start the program.

Step 2: Create a class name as space.

Step 3: Declare the variables integer x,y and z.

Step 4: Define the getdata() function to get the values.

Step 5: Define the display() function to print the values.

Step 6: Define the unary operator_() function to apply the unary operator value to the variable.

Step 7: create a object for class name space.

Step 8: Call the function getdata(),display(),operator_().

Step 9: Display the value.

Step 10: Stop the program.

Program:

```
#include<iostream>
using namespace std;
class space
        {
                int x;
                int y;
                int z;
                public:
                voidgetdata(inta,intb,int c);
                void display();
                void operator-();
        };
void space::getdata(inta,intb,int c)
        {
                x=a;
                y=b;
                z=c;
         }
void space::display()
         {
        cout<<"x="<<x<<" ";
        cout<<"y= "<<y<<" ";
        cout<<"z= "<<z<<"\n";
         }
void space::operator-()
         {
                x=-x;
                y=-y;
                z=-z;
         }
int main()
        {
                space s;
                s.getdata(10,-20,30);
                cout<<"s:";
                s.display();
                -s;
               cout<<"-s:";
                s.display() ;
               return 0;
          }
```

Output:

S: x=10 y= -20 z=30
-S: x= -10 y=20 z= -30

Result:

Thus the program was compiled and executed successfully.

Ex.No: 09 **Demonstrate overloading Binary operator**

Date:01.04.22

Aim:

To write a c++ program demonstrate overloading binary operator.

Algorithm:

Step 1: Start the program.
Step 2: Create a class name as complex.
Step 3: Declare the variables integer a and b.
Step 4: Define the getvalue()function to get the values.
Step 5: Define the binary operator +() overloading function for add the real number and complex number.
Step 6: Define the display() function to print the values.
Step 7: Create two objects for class name complex result.
Step 8: Call the function get value() operator+() and display().
Step 9: Display the value.
Step 10: Stop the program.

Program:

```
#include<iostream>
using namespace std;
class complex
{
inta,b;
public:
void getvalue()
{
        cout<<"Enter the value of complex numbers a,b:";
        cin>>a>>b;
}
complex operator+(complex ob)
{
        complex t;
        t.a=a+ob.a;
        t.b=b+ob.b;
        return(t);
         }
void display()
{
        cout<<a<<"+"<<b<<"i"<<"\n";
}
};
int main()
{
        complex obj1,obj2,result;
        obj1.getvalue();
        obj2.getvalue();
        result=obj1+obj2;
        cout<<"Input values:\n";
        obj1.display();
        obj2.display();
        cout<<"Result:";
        result.display();
        return 0;
 }
```

Output:

Enter the value of complex numbers a,b:2 4

Enter the value of complex numbers a,b:3 4

Input values:

2+4i

3+4i

Result:5+8i

Result:

Thus the program was compiled and executed successfully

Ex.No:10 **Illustrate the multilevel Inheritance**

Date:07.04.22

Aim:

To write a c++ program to multilevel inheritance.

Algorithm:

Step 1: Start the program.

Step 2: Create a baseclass as student.

Step 3: Next create derived class as test.

Step 4: Next create derived class as result.

Step 5: Declare the variables intrno, int marks, float tot, avg.

Step 6: Define the get() functions to get the values.

Step 7: Define the put() functions to print the values.

Step 8: Create object for derived class result.

Step 9: Call the function get(),put() and display().

Step 10: Display the variables.

Step 11: Stop the program.

Program:

```
#include<iostream>
using namespace std;
class student
{
  protected:
  int rno;
  public:
  void getnum()
   {
     cout<<"\nenter the rollnumber:";
     cin>>rno;
   }
   void putnum()
  {
     cout<<"\n roll number:"<<rno;
  }
 };
 class test:public student
 {
  protected:
  int s1,s2,s3;
  public:
  void getmarks()
 {
   cout<<"enter the sub1 marks:";
   cin>>s1;
   cout<<"enter the sub2 marks:";
   cin>>s2;
   cout<<"enter the sub3 mamrks:";
   cin>>s3;
 }
  void putmarks()
 {
 cout<<"\n marks in sub 1:"<<s1;
 cout<<"\n marks in sub 2:"<<s2;
 cout<<"\n marks in sub 3:"<<s3;
 }
 };
class result:public test
{
 float tot,avg;
```

```
    public:
    void display()
    {
    tot=s1+s2+s3;
    avg=tot/3;
    putnum();
    putmarks();
    cout<<"\n total:"<<tot;
    cout<<"\n average:"<<avg;
    }
  };
int main()
{
 result s1;
 s1. getnum();
 s1. getmarks();
 s1.display();
 return 0;
 }
```

Output:

```
enter the rollnumber:116
enter the sub1 marks:96
enter the sub2 marks:86
enter the sub3 mamrks:85

roll number:116
marks in sub 1:96
marks in sub 2:86
marks in sub 3:85
total:267
average:89
```

Result:

Thus the program was compiled and executed successfully

Ex.No:11 **Illustrate the Arithmetic operation on pointers**

Date:09.04.22

Aim:

To write a c++ program to arithmetic operations on pointers.

Algorithm:

Step 1: Start the program.
Step 2: Declare the variables in main function.
Step 3: Get the array value.
Step 4:Assign value p=n.
Step 5: Print the pointer variable of p, p++, p--,p+2, p-2
Step 6: Enter the value.
Step 7: stop the program.

Program:

```
#include<iostream>
using namespace std;
int main()
{
        int n[]={10,25,30,45,35};
        int *p;
        int i;
        cout<<"the array values are:\n";
        for(i=0;i<5;i++)
        {
        cout<<n[i]<<"\n";
         }
         p=n;
          cout<<"\n value of(p):"<<*p;
          cout<<"\n";
         p++;
          cout<<"\n value of p++:"<<*p;
          cout<<"\n";
         p--;
          cout<<"\n value of p--:"<<*p;
          cout<<"\n";
         p=p+2;
          cout<<"\n value of p+2:"<<*p;
          cout<<"\n";
         p=p+3;
          cout<<"\n value of p+3:"<<*p;
          cout<<"\n";
         p=p+4;
          cout<<"\n value of p+4:"<<*p;
          cout<<"\n";
        p=p-2;
         cout<<"\n value of p-2:"<<*p;
         cout<<"\n";
        p=p-3;
         cout<<"\n value of p-3:"<<*p;
         cout<<"\n";
        p=p-4;
         cout<<"\n value of p-4:"<<*p;
         cout<<"\n";
         return 0;
}
```

Output:

The array values are:

10

25

30

45

35

value of(p):10

value of p++:25

value of p--:10

value of p+2:30

value of p+3:0

value of p+4:0

value of p-2:0

value of p-3:35

value of p-4:10

Result:

Thus the program was compiled and executed successfully.

Ex.No:12 **Illustrate the use of two character handling function**.

Date:18.04.22

Aim:

To write a c++ program to illustrate use of two-character handling function.

Algorithm:

Step 1: Start the program.
Step 2: Declare the variable in main()function.
Step 3: Get the upper case string.
Step 4: Convert upper case to lower case using to lower(ch)function.
Step 5: Print the lower case string.
Step 6: Get the lower case string.
Step 7: Convert lower case to uppercase using to upper(ch) function.
Step 8: Print uppercase string.
Step 9: Get the uppercase string.
Step 10: Stop the program.

Program:

```
#include<iostream>
using namespace std;
int main()
{
 int i=0,j=0;
 char str[20],str1[20];
 char ch,ch1;
 cout<<"Enter the upper case string:";
 cin>>str;
 while (str[i])
 {
 ch=str[i];
 putchar(tolower(ch));
 i++;
  }
 cout<<"\nEnter the lowercase string:";
 cin>>str1;
 while(str1[j])
  {
 ch1=str1[j];
 putchar(toupper(ch1));
  j++;
  }
return 0;
 }
```

Output:

```
Enter the upper case string:PROGRAM
                            program
Enter the lowercase string:coding
                           CODING
```

Result:

Thus the program was compiled and executed successfully.

Ex.No: 13 **Create Single file for both writing and reading the data**

Date:20.04.22

Aim:

To write a c++ program a single file for both writing and reading the data.

Algorithm:

Step 1: Start the program.
Step 2: Create a object for fstream.
Step 3: Open a file to write the text.
Step 4: Write text in the file text.txt.
Step 5: Close the file to read the text.
Step 6: Open a file in read mode.
Step 7: Read text from file and set in string variable.
Step 8: Print the text in output.
Step 9: Stop the program.

Program:

```
#include<iostream>
#include<fstream>
Using namespace std;
int main()
{
 fstream ob;
 ob.open("test.txt",ios::out);
 ob<<"hello to all\n";
 ob<<"we are I.BSC information technology";
 ob.close();
 ob.open("test.txt",ios::in);
 while(!ob.eof())
  {
          string str;
          ob>>str;
          cout<<str<<"\n";
    }
    ob.close();
     return 0;
   }
```

Output:

hello
to
all
we
are
I.BSC
information
technology

Result:

Thus the program was compiled and executed successfully.

Ex.No: 14 **Show how a template function is defined and implemented**

Date:02.05.22

Aim:

To write a c++ program to show how a template function is defined and implemented**.**

Algorithm:

Step 1: Start the program.
Step 2: Create a template class as template<class T>
Step 3: Create a class name as number.
Step 4: Define the getnum() function to get the value
Step 5: Declare the object for template class.
Step 6: Declare the variable int a, double b in main()function.
Step 7: Print the integer and double value.
Step 8: Stop the program.

Program:

```
#include<iostream>
using namespace std;
template<class T>
class Number
    {
        private:
        T num;
        public:
        Number(T n):num(n)
        {}
        T getnum()
        {
        return num;
        }
      };
   int main()
   {
                int a;
                double b;
                cout<<"Enter Integer value:";
                cin>>a;
                cout<<"Enter Double value:";
                cin>>b;
                Number<int> numberint(a);
                Number<double> numberdouble(b);
                cout<<"\n Int number="<<numberint.getnum()<<endl;
                cout<<"Double number="<<numberdouble.getnum()<<endl;
                return 0;
      }
```

Output:

```
Enter Integer value:23
Enter Double value:3466

Int number=23
Double number=3466
```

Result:

Thus the program was compiled and executed successfully.

Ex.No: 15 **Illustrate try block throwing an exception**

Date:10.05.22

Aim:

To write a c++ program to illustrate Try block throwing an exception.

Algorithm:

Step 1: Start the program.
Step 2: Declare the variables inta,b.
Step 3: Get the value for a and b.
Step 4: Use try catch and throw.
Step 5: Check condition.
Step 6: Print the value of a and b.
Step 7: Stop the program.

Program:

```
#include<iostream>
using namespace std;
int main()
{
int a,b;
cout<<"Enter values for a and b:";
cin>>a>>b;
int c=a-b;
try
{
if(c!=10)
{
          cout<<"Result:"<<a/c;
}
else
{
        throw(c);
}
}
catch(int i)
{
     cout<<"Exception caught:DIVIDE BY ZERO";
}
          return 0;
}
```

Output:

```
Enter values for a and b:
4
2
Result:2


Enter values for a and b:
5
5
exception caught: DIVIDE BY ZERO
```

Result:

Thus the program was compiled and executed successfully.